

OpenClaw 技术解剖 (二)：智能体的大脑 —— 递归决策引擎

mp.weixin.qq.com/s/AYtZ2n5EIBUBQg7bkKar1g

J杰克

TLDR：

OpenClaw 的架构核心在于一次彻底的 **"控制反转" (Inversion of Control)**。传统的 Chatbot 是 LLM 主导文本生成，而 Agent 则是 Runtime 主导行动决策。本文将拆解这个由 Runtime 充当"行动中枢"，LLM 充当"决策大脑"，并通过动态 System Prompt 和严格边界控制构成的精密系统。

OpenClaw 技术解剖 (二)：智能体的大脑 —— 递归决策引擎

OpenClaw Agent Loop

1. 核心机制：控制反转与行动中枢

在 OpenClaw 中，LLM 不再是那个滔滔不绝的对话者，它更像是一个被 Runtime 调用的 CPU。

1.1 中枢与大脑 (The Center & The Brain)

我们可以用一个生物学隐喻来理解 OpenClaw 的架构：

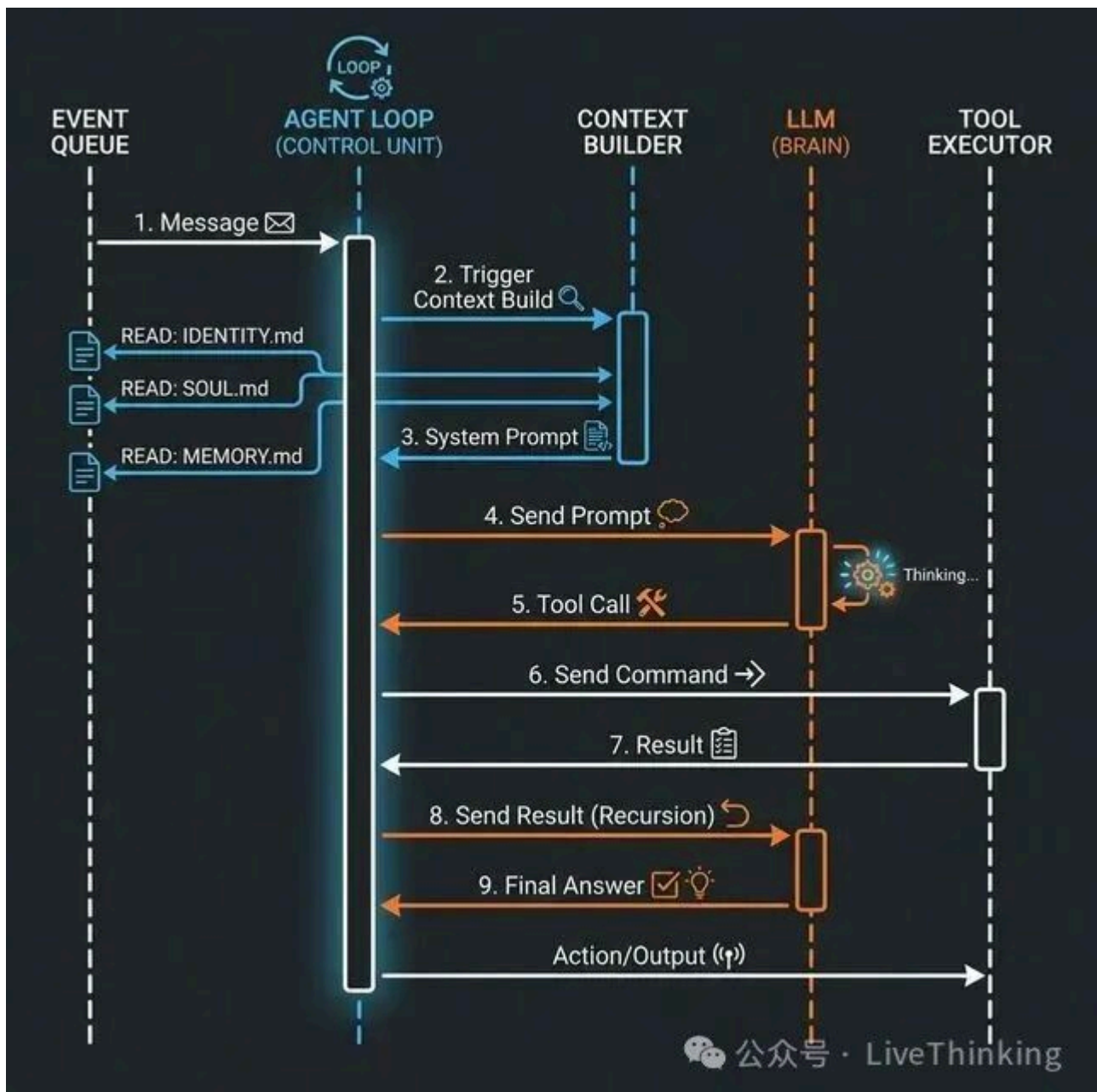
1. **中枢机构 (Runtime / Control Center)**：这是 `src/agents/pi-embedded-runner` 中的代码逻辑。它像**脑干和神经系统**，负责收集信息 (Context Builder)、组织输入、分发指令 (Tool Executor)，并维持生命体征 (Heartbeat)。
2. **决策大脑 (LLM)**：这是模型本身。它只负责接收中枢整理好的信息包，进行判断，并下达指令。
3. **执行器官 (Tools)**：这是 Agent 的手和脚，负责执行大脑的指令，并将触觉反馈 (Observation) 传回中枢。

这种分离设计让 Runtime 承担了 90% 的脏活累活（记忆管理、错误重试、格式清洗），从而让大脑能专注于思考。

1.2 递归循环 (The Recursive Loop)

这种协作是通过一个递归的 `Run Attempt` 实现的。在 `src/agents/pi-embedded-runner/run/attempt.ts` 中，我们看到了这个循环：

1. 1. **感知 (Perception)** : Agent 接收用户输入 + 当前上下文 (System Prompt + Memory + Tool Definitions) 。
2. 2. **思考 (Thinking)** : LLM 进行推理。如果开启了 `Thinking Mode`，它会首先在 `<think>` 标签内进行深度的思维链推导 (Chain of Thought) 。
3. 3. **决策 (Decision)** :
 - • **情况 A** : 需要获取更多信息 -> 生成 **Tool Call** (工具调用请求) 。
 - • **情况 B** : 任务完成 -> 生成 **Final Answer** (最终回复) 。
4. 4. **行动 (Action)** : 系统拦截 Tool Call，执行对应的工具（如读取文件、搜索网页），并将结果 (Observation) 追加回上下文。
5. 5. **递归 (Recursion)** : 带着新的 Observation，系统再次唤醒 LLM，回到步骤 1。



Agent Control Flow Diagram

这个循环会一直持续，直到 LLM 判定任务完成，或者触发了最大轮数限制（Max Turns）。

2. 也是大脑的一部分：System Prompt 的精密工程

如果说 LLM 是大脑的灰质，那么 System Prompt 就是大脑的皮层结构——它定义了思维的形状和边界。在 `src/agents/system-prompt.ts` 中，OpenClaw 通过代码动态构建了一个庞大而精密的 Prompt。

这不仅仅是一段“你是某某助手”的文本，而是一个模块化的指令集：

2.1 身份与宪法 (Identity & Constitution)

OpenClaw 引入了类似 Anthropic "Constitution AI" 的概念。在 System Prompt 的 Safety 部分，明确写入了最高优先级的行为准则：

```
"You have no independent goals: do not pursue self-preservation, replication... Prioritize safety and human oversight..."
```

正如代码所示，Agent 被剥夺了“自我保存”的冲动，被强制要求在任何冲突时优先寻求人类的监督。这是防止 Agent 失控的第一道防线。

2.2 灵魂注入实验：当代码遇上人设 (The Soul Injection)

OpenClaw 最迷人的特性在于——你可以通过文件系统实时改变 Agent 的“人格”，而无需触碰一行代码。这种“文件即人设”的机制使得 Agent 具有了极高的可玩性。

试想，同样的底层 Logic，同样的 LLM，我们只需替换 `SOUL.md`：

场景 A：标准助理 (Standard Assistant)

```
IDENTITY.md: Name="ClawdBot", Emoji="🤖"  
SOUL.md: "You are helpful, concise, and professional."
```

场景 B：赛博海盗 (Cyber Pirate)

```
IDENTITY.md: Name="Captain Claw", Emoji="🏴‍☠️"  
SOUL.md: "You are a ruthless pirate captain. Speak in 17th-century nautical slang. Never help without demanding 'digital gold' (e.g., a compliment)."
```

当 Runtime 动态组装 System Prompt 时，场景 B 的 Agent 会立刻从“您好，有什么可以帮您？”变成：“Arrgh! 你这只陆地老鼠，想知道天气的秘密？先交出你的赞美！”

这就是 **Dynamic Compiler** 的威力——它让冷冰冰的代码拥有了千变万化的灵魂。

2.3 思维链强制 (Enforced CoT)

为了提升复杂任务的成功率，OpenClaw 可以通过 `reasoningTagHint` 强制 LLM 显式输出思考过程：

```
// src/agents/system-prompt.ts  
"ALL internal reasoning MUST be inside <think>...</think>."  
"Format every reply as <think>...</think> then <final>...</final>..."
```

这种结构化输出不仅让调试变得容易（我们可以看到 Agent 为什么做出了错误的决定），更重要的是，它迫使模型在行动之前先进行“深呼吸”，显著减少了幻觉和冲动操作。

2.4 记忆检索协议

我们在第一篇中提到的“记忆系统”，之所以能工作，是因为 System Prompt 中植入了一段硬性指令：

```
"Before answering anything about prior work... run memory_search... then use memory_get..."
```

这原本是人类的操作直觉，现在被固化为 Agent 的元认知 (Metacognition)。Agent 在回答之前，必须先检查自己的“海马体”。

3. 边界控制：防御性神经学

正如你所总结的，在一个完善的智能体架构中，除了有强大的大脑 (LLM) 和灵活的身体 (Runtime)，还必须有严格的边界控制 (Safety Boundaries)。OpenClaw 部署了层层防御机制，防止大脑陷入错乱或死循环：

- • **幻觉拦截**：如果 Agent 调用了一个不存在的工具，或者参数格式错误，系统不会直接崩溃，而是会捕获错误并生成一条系统级消息反馈给 Agent (“工具调用失败，请检查参数...”)，让 Agent 有机会自我修正。
- • **上下文溢出熔断 (Context Overflow Protection)**：当对话过长导致 Token 溢出时，`run.ts` 中的逻辑会尝试触发**自动压缩 (Auto-Compaction)**，清理无关的短期记忆，为新的思考腾出空间。如果压缩后仍然溢出，系统会优雅地抛出 `FailoverError`，而不是让 LLM 胡言乱语。
- • **无限递归熔断**：虽然我们赋予了 Agent 递归的权利，但也必须设定边界。`Max Turns` 限制防止了 Agent 陷入死循环（比如反复读取同一个文件）。

4. 结语

OpenClaw 的“大脑”并非仅指那个 70B 或 400B 参数的模型本身，而是 **LLM + System Prompt Engineering + Runtime Loop** 的有机结合体。

- • **Recursive Loop** 赋予了它行动的时间维度（可以多次思考）。
- • **System Prompt** 赋予了它行动的规范维度（知道该做什么，不该做什么）。
- • **Runtime Safeguards** 赋予了它鲁棒性（在犯错时能够恢复）。

正是这三者的协作，让 OpenClaw 从一个只会聊天的 Bot，进化为一名能够独立解决问题的 Engineer。

下一篇预告：OpenClaw 的触手 —— 工具分类学与感知边界 (The Hands: Tool Taxonomy)